



ScholarRAG

An AI-Powered Academic Research Assistant

Retrieval-Augmented Generation over 22,000+ Research Papers, Containerized and Orchestrated on Kubernetes

Domain	Cloud Computing, DevOps & Generative AI (RAG)
Mentor	Mr. Ujjwal Pratap Singh — Founder, Skill Nebula
Period	13 June 2026 to 28 June 2026

Team Members & Contributions

Name	Role	Key Contribution
Ms. Akriti Yadav	Team Leader	Dataset collection & preparation, Docker image build/versioning, Kubernetes YAML manifests
Mr. Oum Gupta	DevOps + RAG/API	kind cluster & Ingress, K8s manifests, NGINX SSE routing, Secrets; RAG pipeline & FastAPI streaming API
Mr. Anurag Kashyap	DevOps + Data	JSON→SQLite migration, NGINX reverse proxy, Docker containerization, multi-tier K8s deployment
Ms. Shruti Gupta	Frontend	React SPA UI/UX, responsive design, REST API integration with the FastAPI backend
Mr. Saurabh Chandra Shukla	Frontend + Infra	React frontend development and Kubernetes deployment YAML manifests

Declaration

The project titled “ScholarRAG — An AI-Powered Academic Research Assistant” was built as part of a Cloud Computing internship mentored by Mr. Ujjwal Pratap Singh, Founder of Skill Nebula. During the internship he trained the team in modern cloud technologies — AWS, Docker and Kubernetes — and set a 15-day challenge (13 June 2026 to 28 June 2026) to design, containerize and deploy this application end-to-end. We declare that to the best of our knowledge the members of the team mentioned below have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members	Mentor
Ms. Akriti Yadav (Team Leader) Mr. Oum Gupta Mr. Anurag Kashyap Ms. Shruti Gupta Mr. Saurabh Chandra Shukla	Mr. Ujjwal Pratap Singh Founder, Skill Nebula Mentor & Program Lead

Table of Contents

1.	Executive Summary
2.	Introduction
3.	Project Objectives
4.	Key Features
5.	System Architecture & Container Communication
6.	Methodology & Results
7.	Final Output (Screenshots)
8.	Social / Industry Relevance
9.	Learning & Reflection
10.	Future Scope & Conclusion
11.	Appendix — Deployment Commands & Code Snippets

1. Executive Summary

Objective

Build ScholarRAG, an AI-powered research assistant that lets users ask natural-language questions and upload research PDFs, then returns grounded, citation-backed answers drawn from a corpus of 22,000+ academic papers. The complete stack is containerized with Docker and deployed on a multi-node Kubernetes (kind) cluster fronted by an NGINX Ingress.

Methods

A FastAPI backend implements a Retrieval-Augmented Generation (RAG) pipeline: HyDE query expansion → bi-encoder semantic search over a Qdrant vector database → cross-encoder reranking → answer generation with Anthropic Claude, streamed to the browser over Server-Sent Events (SSE). A React single-page frontend, a SQLite user/chat store, Docker images per service, and Kubernetes manifests (Deployments, Services, Ingress) complete the 3-tier system.

Key Findings

- The full RAG pipeline works end-to-end: HyDE expansion, embedding with all-MiniLM-L6-v2, vector retrieval from the papers collection in Qdrant, and cross-encoder reranking with ms-marco-MiniLM-L-6-v2 before Claude generates the final answer.
- The API surface is clean and REST-based under FastAPI: `/register`, `/login`, `/query`, streaming `/query/stream` (SSE), chat management (`/chats`, `pin`, `star`, `delete`, `clear`) and a `/health` probe.
- A tunable relevance floor (`RERANK_FLOOR`, env-configurable) filters weak matches, with a separate lower floor for uploaded-PDF queries so on-topic papers are never dropped.
- Kubernetes manifests run the backend (2 replicas, port 8000), frontend (2 replicas, port 80) and Qdrant (port 6333); the NGINX Ingress routes `/api/*` to the backend and everything else to the frontend, with proxy buffering disabled so SSE streams token-by-token.
- Secrets (Anthropic API key, tokens) are injected via Kubernetes Secrets; the SQLite database is persisted through a host-path mount so user accounts and chats survive pod restarts.

Recommendations

- Replace host-path storage for SQLite with a managed PVC (or migrate to a managed Postgres) for true multi-node durability.
- Add liveness probes and CPU/memory requests & limits to every Deployment alongside the existing readiness probe.
- Introduce rate-limiting and stronger input validation on the query endpoints, and move Qdrant to a persistent volume so the vector index is not rebuilt on restart.
- Add a CI/CD pipeline that builds and pushes versioned images and performs rolling updates automatically, plus Prometheus/Grafana observability for latency and token usage.

2. Introduction

Background and Context

The volume of academic literature grows faster than any researcher can read. Keyword search returns links, not answers, and large language models on their own hallucinate citations. ScholarRAG was built as a cloud-native solution to this problem: it combines a curated corpus of 22,000+ research papers with Retrieval-Augmented Generation so that every answer is grounded in real, retrievable sources. By packaging the retrieval service, generation service, vector database and frontend as independent containers and orchestrating them with Kubernetes, the project shows how a modern AI application can be made reproducible, scalable and highly available.

Problem Statement and Goals

Researchers waste hours locating and cross-reading papers, and general chatbots cannot be trusted to cite accurately. The primary goals of this project were:

- **Grounded Answers:** Provide a single interface where a natural-language question returns a synthesized answer backed by actual papers and, where available, linked GitHub implementations.
- **Retrieval Quality:** Combine HyDE query expansion, bi-encoder retrieval and cross-encoder reranking so the most relevant papers surface first instead of the most keyword-similar ones.
- **DevOps Integration:** Deploy the entire stack — Frontend, Backend, and Vector Database — into a multi-node Kubernetes cluster, bridging the gap between AI development and operations.

Scope and Limitations

- **Scope:** User authentication, a streaming RAG query engine, PDF upload and analysis, persistent chat history, and a fully containerized deployment orchestrated on a kind Kubernetes cluster behind an NGINX Ingress.
- **Security:** Secrets are stored in Kubernetes Secrets, but the current build uses session-token auth suitable for demonstration rather than a hardened production identity provider.
- **Persistence:** SQLite is persisted via a host-path mount and Qdrant currently runs a single replica — adequate for the cluster demo but not for large-scale high availability.
- **Scale:** The system is optimized for a local, multi-node kind environment rather than a managed public-cloud cluster such as EKS or AKS.

Innovation Component

- **Two-Stage Retrieval:** A fast bi-encoder narrows 22k papers to a candidate set, then a cross-encoder re-scores each candidate against the query for precision — a technique used in production search systems, applied here end-to-end.
- **HyDE Query Expansion:** The system asks the LLM to draft a hypothetical answer and embeds that for search, closing the vocabulary gap between short questions and dense abstracts; results are cached for determinism and speed.
- **Streaming, Self-Healing Delivery:** Answers stream token-by-token over SSE through an Ingress tuned to disable buffering, while Kubernetes provides self-healing and rolling updates across a three-node cluster.

3. Project Objectives

Clearly Defined Objectives and Goals

The primary objective was to engineer a robust, containerized AI application that answers research questions through a grounded, streaming pipeline. Specifically, the project aimed to:

- **Implement a Multi-Tier Architecture:** A decoupled system where the React frontend, FastAPI backend, and Qdrant vector database operate as independent, separately scalable services.
- **Build a High-Quality RAG Pipeline:** Combine HyDE expansion, semantic vector search and cross-encoder reranking with a tunable relevance floor to maximize answer grounding.
- **Master Container Orchestration:** Move beyond localhost by deploying the whole stack into a multi-node Kubernetes (kind) cluster, focusing on service discovery, Ingress routing and rolling updates.
- **Stream Responses in Real Time:** Deliver answers token-by-token over Server-Sent Events so the user sees the model think, with the Ingress configured to pass streams through unbuffered.
- **Secure & Persist State:** Inject API keys via Kubernetes Secrets and persist user accounts and chat history so they survive pod restarts.

Expected Outcomes and Deliverables

Technical Deliverables

- **Source Code:** A React frontend and a FastAPI backend implementing the full RAG pipeline (rag.py) and REST API (main.py).
- **Ingestion Pipeline:** Scripts that fetch, filter and resample 362k raw records down to a curated 22k-paper corpus (10k Papers-with-Code + 12k OpenAlex) and embed them into Qdrant.
- **Container Images:** Versioned Docker images per service (scholarrag:backend-v27, frontend-v7, qdrant-v1).
- **Orchestration Manifests:** A complete set of Kubernetes YAML under ./k8s defining Deployments, Services, an NGINX Ingress, a Secret, and the kind cluster configuration.
- **RESTful API:** A structured surface handling auth, streaming queries, PDF upload and chat management.

Expected Outcomes

- **Grounded, Cited Answers:** Responses backed by real papers and, when relevant, links to their GitHub implementations — not unverifiable free text.
- **Deployment Portability:** A “write once, run anywhere” setup whose manifests can be pushed to a managed cloud cluster (EKS/AKS) with minimal change.
- **Resilience:** Two backend and two frontend replicas with rolling updates and a readiness probe so the service stays available during redeployments.

4. Key Features

ScholarRAG combines a production-grade RAG pipeline with a cloud-native deployment. Its principal features are:

Feature	Description
Grounded RAG Answers	Every response is generated by Anthropic Claude using only retrieved papers as context, with inline citations — directly countering LLM hallucination.
HyDE Query Expansion	Claude drafts a hypothetical answer which is embedded for search, closing the vocabulary gap between short questions and dense abstracts (cached for speed).
Two-Stage Retrieval	A bi-encoder (all-MiniLM-L6-v2) narrows 22k papers to 25 candidates; a cross-encoder (ms-marco-MiniLM) reranks them to the best 3.
Anti-Hallucination Floor	A tunable RERANK_FLOOR drops weak matches; if nothing qualifies, no sources are shown rather than forcing an irrelevant paper.
Real-Time Streaming	Answers stream token-by-token over Server-Sent Events, with live status events (“Searching...” → “Generating...”).
Conversational Memory	Per-session chat history is replayed into every request so follow-up questions retain context.
PDF Upload & Analysis	Uploaded PDFs are parsed with PyMuPDF, abstract-anchored for retrieval, and answered against specifically.
Agentic Tool Use	A fetch_url tool lets Claude read a paper's GitHub README or arXiv page on demand for deeper detail.
Source Citations Panel	Each answer returns titles, authors, year, arXiv/PDF links, GitHub repos and a relevance score.
Cloud-Native Deployment	Dockerized services on a multi-node kind cluster with an NGINX Ingress, rolling updates and a readiness probe (zero-downtime).
Secure Secrets	Anthropic / GitHub / HuggingFace keys injected via Kubernetes Secrets — never hard-coded in images.
Persistent State	User accounts and chats stored in SQLite, persisted through a host-path mount so they survive pod restarts.

5. System Architecture & Container Communication

Architecture Overview

ScholarRAG follows a cloud-native 3-tier architecture running inside a Kubernetes (kind) cluster of one control-plane and two worker nodes, all hosted on a single EC2 instance. External traffic enters on port 8080, is mapped by kind into the cluster, and the NGINX Ingress routes each request to the correct tier.

- **Client Tier (Frontend):** A React SPA served by two frontend pods on port 80.
- **Application Tier (Backend + RAG):** Two FastAPI pods on port 8000 run the RAG pipeline and business logic (auth, retrieval, reranking, streaming generation).
- **Data Tier:** A Qdrant pod (port 6333) holds the paper embeddings; a SQLite database persisted via a host-path mount stores users and chat history; Anthropic Claude is called as an external LLM API.

Kubernetes Deployment Architecture

The whole system runs inside one EC2 server. kind maps host port 8080 to the cluster; the Ingress controller is pinned to the control-plane node, Services provide stable cluster-wide addresses, and the application pods are scheduled across the two worker nodes.

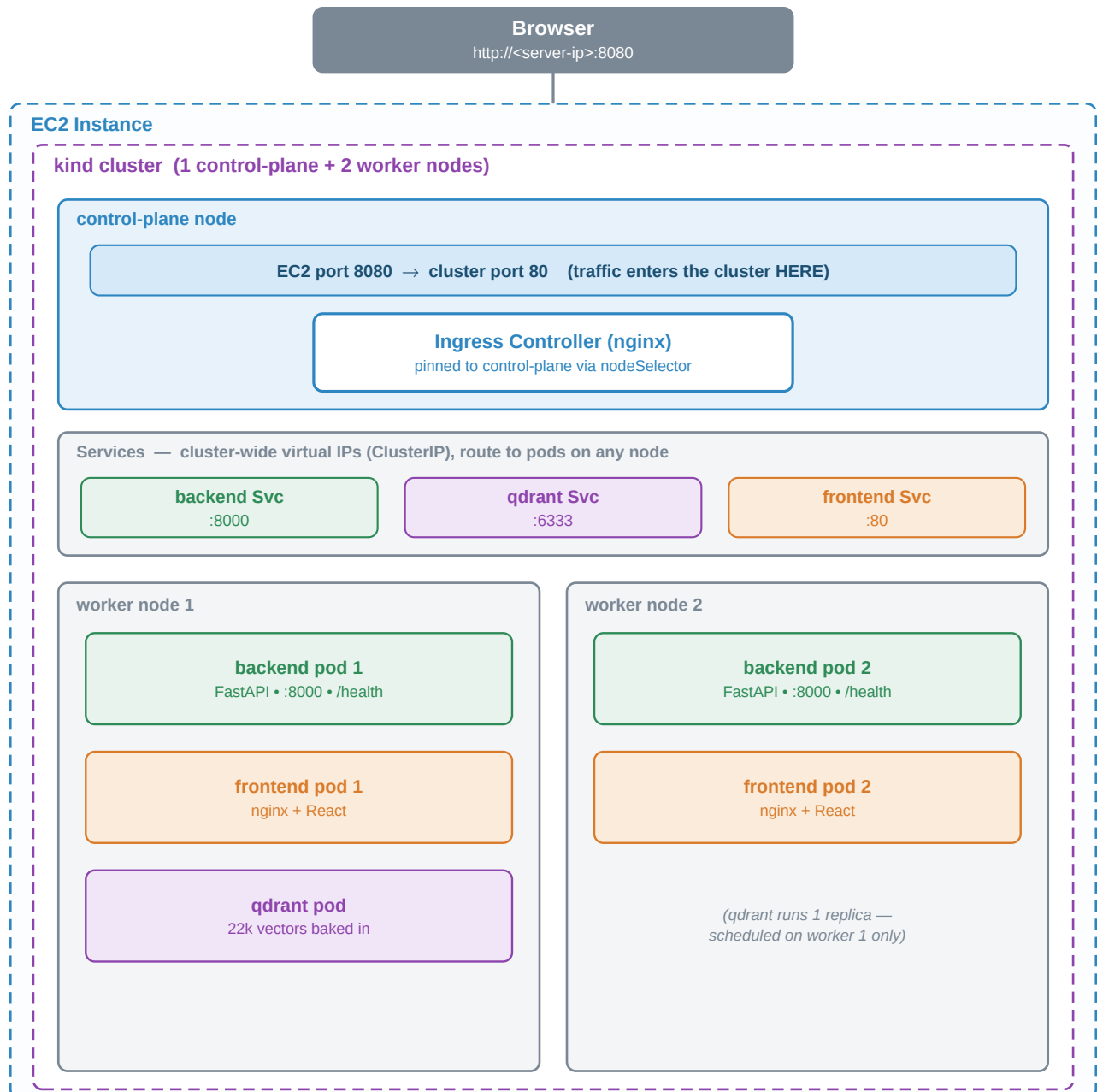
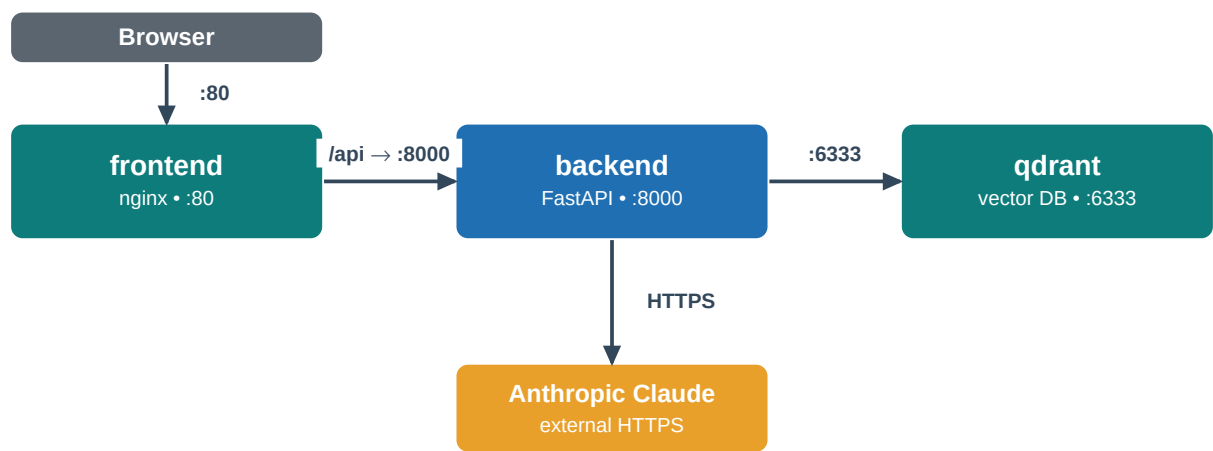


Figure 1: Kubernetes architecture at a glance — a single EC2 running the kind cluster (1 control-plane + 2 workers). Traffic enters at the control-plane, Services route to pods on any node, and the app pods run on the worker nodes.

How the Containers Talk to Each Other

Each service reaches the next one **by name** — never by IP. On Docker Compose the service names resolve over the project network; on Kubernetes the same names are exposed as Services and resolved by the cluster's built-in DNS, which also load-balances across replicas.



Each name (frontend / backend / qdrant) resolves via the network DNS — no IP addresses are used.

Figure 2: Inter-container communication and the port each service exposes.

Caller	Callee	Address used	Purpose
Browser	Frontend (nginx)	<host>:80 / :8080	Load the React UI
Frontend (nginx)	Backend	backend:8000	Proxy /api/* calls
Backend	Qdrant	qdrant:6333	Vector similarity search
Backend	Anthropic Claude	api.anthropic.com (HTTPS)	Answer generation
Backend	SQLite	/app/data/users.db (mount)	Users & chat history

Key idea: the frontend calls backend:8000, the backend calls qdrant:6333, and the platform (Docker network DNS or Kubernetes Service DNS) turns each name into the right container/pod — so replicas can scale and restart with new IPs without any config change.

6. Methodology and Results

Methods & Technology Used

The project followed an Agile methodology, splitting the system into independent components (Frontend, Backend/RAG, Vector Database, Orchestration) so each could be developed, dockerized and tested in isolation.

- **Data Ingestion:** Research papers were fetched from Papers-with-Code and OpenAlex, deduplicated, resampled and embedded with the all-MiniLM-L6-v2 sentence-transformer into a Qdrant collection.
- **Backend & RAG:** Built with FastAPI. The pipeline runs HyDE expansion → vector search → cross-encoder rerank → Claude generation, with an agentic fetch_url tool that reads GitHub READMEs and arXiv pages on demand.
- **Frontend:** A React SPA with authentication, multi-chat history, drag-and-drop PDF upload and live token streaming.
- **Containerization & Orchestration:** Every service is dockerized; Kubernetes (kind) manages deployment, replicas, service discovery and Ingress routing.

The RAG Pipeline



Figure 3: The retrieval-augmented generation pipeline, question to streamed answer.

Tools & Software Used

Category	Tools
Backend & API	Python, FastAPI, Uvicorn
RAG & AI	Anthropic Claude, HyDE, HuggingFace all-MiniLM-L6-v2 (bi-encoder), cross-encoder ms-marco-MiniLM-L-6-v2, PyMuPDF
Vector Database	Qdrant (“papers” collection)
Frontend	React, JavaScript, Server-Sent Events (SSE)
Data Store	SQLite (users & chat history)
DevOps & Infra	Docker, Kubernetes (kind), kubectl, NGINX Ingress, AWS EC2
Version Control	Git & GitHub

7. Final Output (Screenshots)

The screenshots below show ScholarRAG running end-to-end on the deployed application — served both by Docker Compose (port 80) and by the Kubernetes cluster (port 8080).

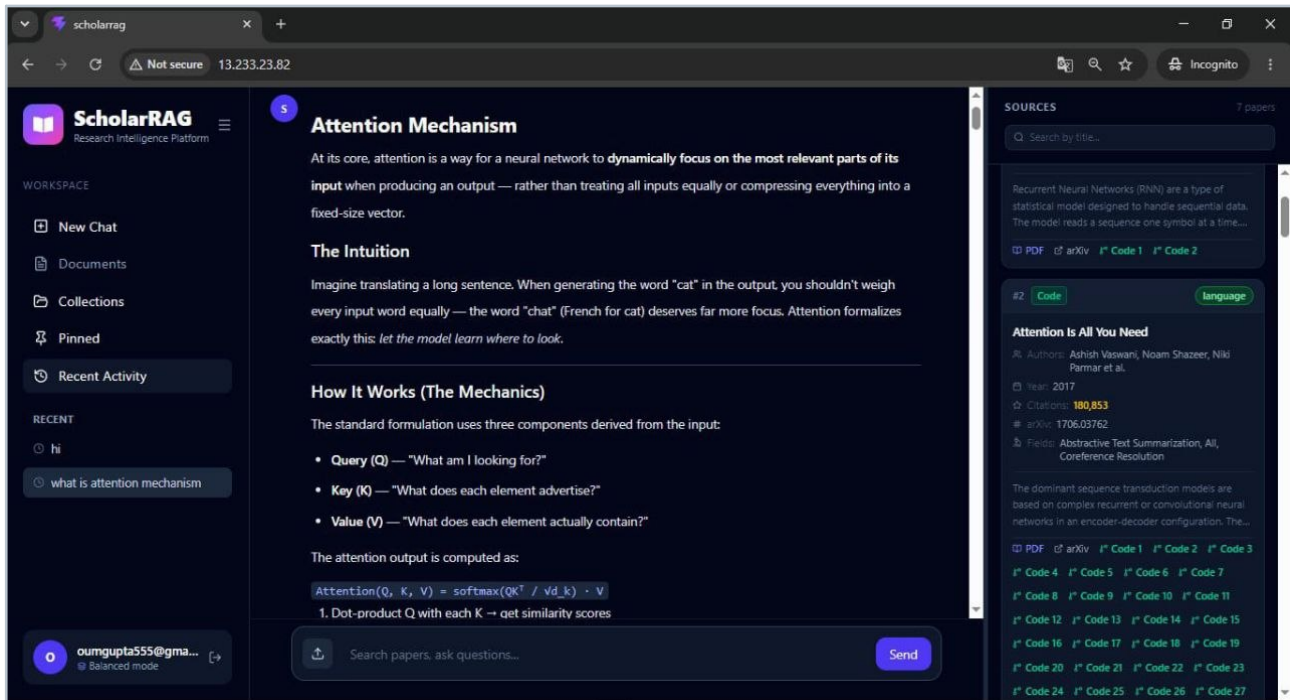


Figure 4: RAG query with a streamed, grounded answer and the live Sources/citations panel (served via Docker Compose, port 80). The question “what is attention mechanism” returns a synthesized explanation with the “Attention Is All You Need” paper cited on the right.

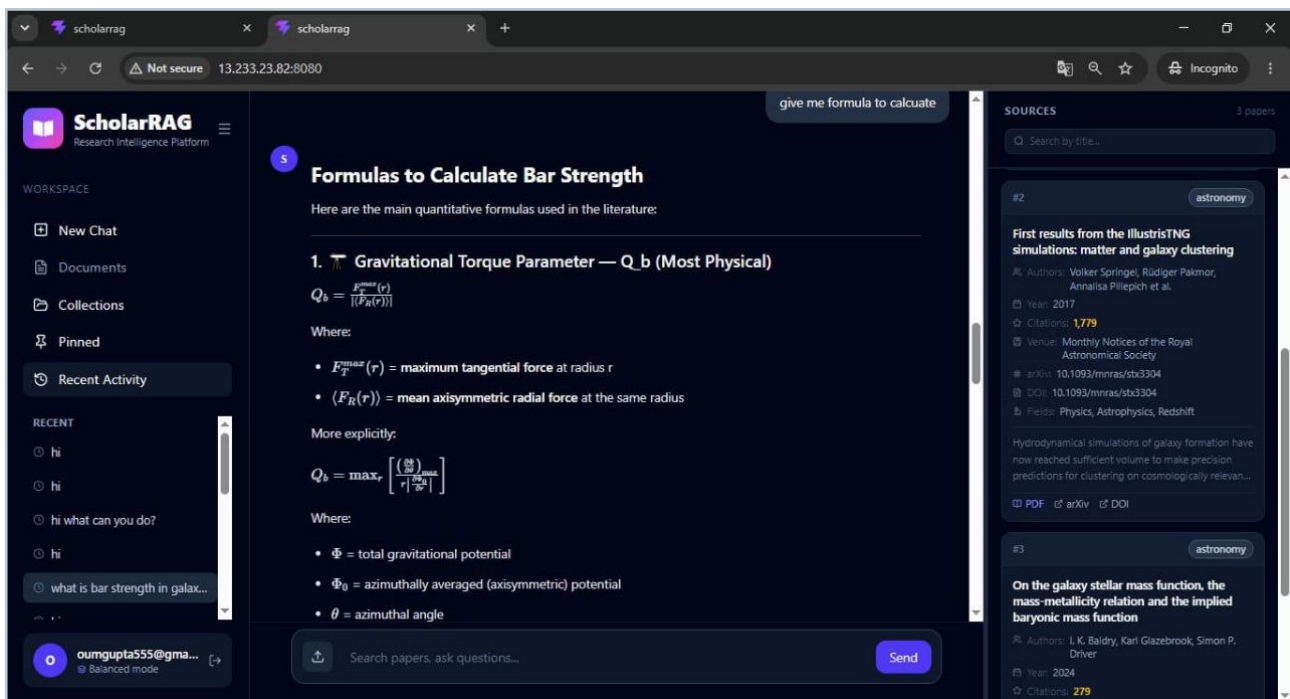


Figure 5: The same application served by the multi-node Kubernetes cluster (port 8080), answering a domain question (bar-strength formula) with astronomy papers retrieved and cited — confirming the Kubernetes deployment works end-to-end.

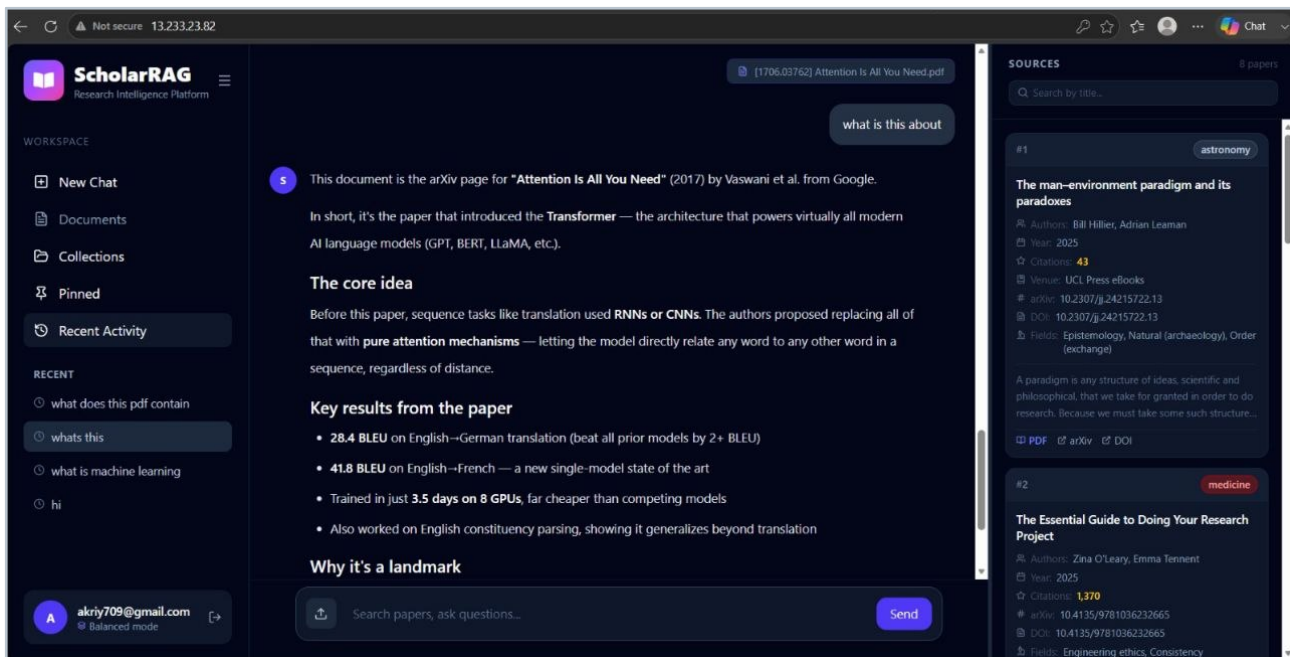


Figure 6: PDF upload and document-grounded analysis. The uploaded “Attention Is All You Need.pdf” is parsed with PyMuPDF and the model answers specifically about that document, with related papers listed as sources.

Project GitHub Link

The full source code — ingestion scripts, backend RAG pipeline, React frontend, Dockerfiles and Kubernetes YAML manifests — is available in the project repository on GitHub: github.com/Akriti-236/ScholarRAG.

8. Social / Industry Relevance of the Project

ScholarRAG sits at the intersection of two of the most in-demand areas in industry today: applied Generative AI and cloud-native DevOps. Its relevance spans several fronts:

- **Democratizing Research:** Students and independent researchers — including those in rural and under-resourced settings — gain a free, grounded way to explore 22k+ papers without expensive journal tooling.
- **Trustworthy AI:** By grounding every answer in retrievable sources, the project directly addresses hallucination, the single biggest barrier to adopting LLMs in education and enterprise.
- **Industry-Standard Stack:** FastAPI, Qdrant, Docker, Kubernetes and Ingress-based routing are exactly the technologies used to ship real AI products, making the skills gained immediately employable.
- **Reusable Pattern:** The same retrieval-plus-orchestration blueprint applies to legal, medical and internal-knowledge assistants, so the work generalizes well beyond academic papers.

9. Learning and Reflection

Reflections on Technology

The leap from running the pipeline on localhost to running it inside a Kubernetes cluster was the team's biggest milestone. We saw first-hand that Docker makes an app portable, but Kubernetes makes it production-ready — self-healing pods, rolling updates and Ingress routing turned a script into a service. Getting SSE streaming to survive the Ingress (by disabling proxy buffering) was a genuine “lightbulb” moment about how real infrastructure shapes application behaviour.

Reflections on Collaboration

Working as a team demanded discipline with Git and with shared configuration. We learned that a single changed service name or environment variable in a Kubernetes manifest can break the backend's connection to Qdrant or the database, so we relied on regular sync-ups and clear ownership of the frontend, backend and infra layers to keep everything integrated.

Challenges Overcome

- **Streaming through the Ingress:** Responses buffered instead of streaming until we disabled proxy buffering and raised the read/send timeouts on the NGINX Ingress.
- **Retrieval relevance:** Early answers pulled in off-topic papers; adding cross-encoder reranking with a tunable RERANK_FLOOR (and a separate PDF floor) fixed grounding.
- **Persisting state in a cluster:** User accounts and chats were lost on restart until we mounted the SQLite database via a host-path volume and injected keys through Kubernetes Secrets.
- **Image and env collisions:** Debugging ImagePullBackOff and a QDRANT_PORT env clash taught us how images are pulled and how env precedence works in Kubernetes.

Individual Experience Summaries

- **Akriti Yadav (Team Leader)**

“Serving as Team Leader for ScholarRAG was both challenging and rewarding. I assigned responsibilities, tracked progress and kept the team aligned to our deadlines. My technical contributions included collecting research papers from multiple sources and preparing the dataset for the RAG pipeline, creating and versioning the Docker images, and writing Kubernetes YAML manifests for deployment. Balancing leadership with hands-on DevOps work strengthened my communication, decision-making and project-management skills.”

- **Oum Gupta**

“I worked on the Kubernetes orchestration, the RAG pipeline and the API layer. On the infrastructure side I set up the multi-node kind cluster, wrote the Deployment, Service and Ingress manifests, configured NGINX to route /api traffic to the backend and to stream SSE responses unbuffered, and wired API keys through Kubernetes Secrets with rolling updates for zero-downtime redeploy. On the RAG side I built the retrieval pipeline — HyDE query expansion, bi-encoder search over Qdrant and cross-encoder reranking with a tunable relevance floor — and exposed it through the FastAPI endpoints, including the streaming /query/stream route that the frontend calls.”

- **Anurag Kashyap**

“As a member of the ScholarRAG team I contributed to building and deploying a scalable cloud-based application. My responsibilities included migrating data from JSON to SQLite, configuring NGINX as a reverse proxy, containerizing the application with Docker, and deploying the complete multi-tier application on Kubernetes. I collaborated closely with teammates to troubleshoot deployment issues, improve the

architecture and automate the workflow, strengthening my knowledge of Docker, Kubernetes, NGINX, SQLite, AWS and DevOps practices.”

- **Shruti Gupta**

“During the ScholarRAG project I served as a Frontend Team Member, developing the web application's frontend in React with a strong emphasis on responsive design, usability and clean UI. I integrated the frontend seamlessly with the FastAPI backend, gaining hands-on experience in REST API integration, reusable components and application performance. I also built a strong understanding of Retrieval-Augmented Generation, Large Language Models and context-aware response generation.”

- **Saurabh Chandra Shukla**

“I worked as a frontend developer and wrote the YAML manifests used for our Kubernetes deployment. Through this I gained hands-on experience integrating Kubernetes configuration into a real RAG-based system and understanding how the deployment pieces fit together.”

10. Conclusion and Future Scope

Recap of Objectives and Achievements

The project successfully delivered a grounded, cloud-native AI research assistant. By the end of the development cycle the team achieved the following:

- **Working RAG Pipeline:** A full HyDE → bi-encoder → cross-encoder → Claude pipeline over 22k+ papers, returning citation-backed answers streamed in real time.
- **Successful Orchestration:** A multi-node Kubernetes (kind) cluster running frontend, backend and Qdrant behind an NGINX Ingress, with rolling updates and a readiness probe.
- **Reliable State & Secrets:** Persistent user and chat data and API keys injected securely via Kubernetes Secrets.
- **Team Synergy:** A practical laboratory for mastering Dockerization, Kubernetes manifests, REST API design and modern RAG techniques in a collaborative setting.

Future Scope

1. Retrieval & Model Quality

- Expand the corpus beyond 22k papers and add hybrid (dense + keyword) search for higher recall.
- Add answer-level citation highlighting and confidence scoring so users can see exactly which sentence came from which paper.

2. Advanced Infrastructure

- Move SQLite and Qdrant to managed persistent volumes (or Postgres / Qdrant Cloud) for true high availability, and add Horizontal Pod Autoscaling on the backend.
- Deploy to a managed cloud cluster (AWS EKS / Azure AKS) with TLS termination at the Ingress.

3. Observability & CI/CD

- Integrate Prometheus and Grafana to monitor pod health, query latency and token usage.
- Establish a CI/CD pipeline (GitHub Actions) so every push builds a versioned image and triggers a rolling update automatically.

4. Expanded Features

- Add user-facing citation export (BibTeX) and saved-search alerts for newly ingested papers.
- Support multi-document PDF sessions and follow-up question threads grounded in the uploaded corpus.

11. Appendix — Deployment Commands & Code Snippets

This appendix lists the key commands used to run ScholarRAG and to deploy it to the Kubernetes cluster, along with representative code snippets from the RAG pipeline and API.

A. Docker — build, push and run the stack

```
# build, tag and push the three service images
docker build -t akritiyadav/schollarrag:backend-v27 ./backend
docker push akritiyadav/schollarrag:backend-v27
docker build -t akritiyadav/schollarrag:frontend-v7 ./frontend
docker push akritiyadav/schollarrag:frontend-v7

# run the whole stack locally (frontend on http://localhost:80)
docker compose up -d
docker compose logs -f backend
```

B. Kubernetes (kind cluster) — deploy in order

```
# 1. create the 1 control-plane + 2 worker cluster
kind create cluster --name schollarrag --config k8s/kind-config.yaml
# 2. install the NGINX ingress controller, then wait for it
kubectl apply -f k8s/ingress-controller.yaml
kubectl wait --namespace ingress-nginx --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller --timeout=90s
# 3. inject secrets from the .env file
kubectl create secret generic schollarrag-secret --from-env-file=.env
# 4. deploy the three tiers, then the routing rules
kubectl apply -f k8s/qdrant-deployment.yaml
kubectl apply -f k8s/backend-deployment.yaml
kubectl apply -f k8s/frontend-deployment.yaml
kubectl apply -f k8s/ingress.yaml
```

C. Verify, roll out and tear down

```
kubectl get pods -o wide # all pods Running / Ready
curl http://localhost:8080/api/health # {"status":"ok","version":"v27"}
# ship a new backend image with zero downtime (maxUnavailable: 0)
kubectl set image deployment/backend backend=akritiyadav/schollarrag:backend-v27
kubectl rollout status deployment/backend
kind delete cluster --name schollarrag # full teardown
```

D. Code Snippet — two-stage retrieval (backend/rag.py)

```
def _search(query_text, k=5):
    vector = _embeddings.embed_query(query_text) # all-MiniLM-L6-v2
    return _qdrant.query_points(collection_name=COLLECTION,
                                query=vector, limit=k).points

def _rerank(query, candidates, top_n):
    pairs = [[query, r.payload['title']] + ' ' + r.payload['summary']]
    for r in candidates]
    scores = _cross_encoder.predict(pairs) # ms-marco-MiniLM
    ranked = sorted(zip(candidates, scores), key=lambda cs: cs[1], reverse=True)
    return [(c, float(s)) for c, s in ranked[:top_n]]
```